

Modulo 1B — Funzioni, Case, Test Avanzati

Corso: Lab Amministrazione di Sistemi T

Prerequisito: Modulo 1A completato (variabili, if/else, loop, exit code)

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Contesto

Nel Modulo 1A hai costruito script che fanno una cosa sola, in sequenza. In questo modulo impari a strutturare script più complessi usando **funzioni** — blocchi di codice riutilizzabili con un nome — e il costrutto **case**, che sostituisce le catene di `if/elif` quando i confronti sono su un singolo valore. Impari anche a combinare condizioni con gli operatori logici e a leggere l'exit code di qualsiasi comando con `$?`.

Crea la directory di lavoro:

```
mkdir ~/Lab1B && cd ~/Lab1B
```

Parte 1 — Funzioni

1.1 Dichiarazione e chiamata

Una funzione è un blocco di comandi a cui assegni un nome. Lo chiami più volte senza riscrivere il codice.

```
cat > funzione_base.sh << 'EOF'
#!/bin/bash

saluta() {
    echo "Ciao, sono la funzione saluta"
    echo "La data è: $(date)"
}

echo "Prima della chiamata"
saluta
echo "Dopo la chiamata"
saluta
EOF
chmod +x funzione_base.sh
./funzione_base.sh
```

Output atteso:

```
Prima della chiamata
Ciao, sono la funzione saluta
La data è: [data corrente]
Dopo la chiamata
Ciao, sono la funzione saluta
La data è: [data corrente]
```

La funzione viene **definita** (`saluta() { ... }`) e poi **chiamata** scrivendo il suo nome (`saluta`). La definizione deve precedere la chiamata nello script. Le due sintassi equivalenti per dichiarare una funzione sono:

```
saluta() { ... }           # forma compatta - quella più usata
function saluta() { ... } # forma esplicita - identica, solo più verbosa
```

1.2 Argomenti di una funzione

All'interno di una funzione, `$1`, `$2`, `$#`, `$0` si riferiscono agli argomenti **della funzione**, non dello script. È una distinzione critica.

```
cat > funzione_argomenti.sh << 'EOF'
#!/bin/bash

descrivi_file() {
    FILE=$1
    if [ -z "$FILE" ]; then
        echo "Errore: nessun file specificato"
        return 1
    fi
    if [ -f "$FILE" ]; then
        echo "$FILE: è un file regolare"
    elif [ -d "$FILE" ]; then
        echo "$FILE: è una directory"
    else
        echo "$FILE: non esiste"
    fi
}
```

```
descrivi_file /etc/hostname
descrivi_file /etc
descrivi_file /file_inesistente
descrivi_file
EOF
chmod +x funzione_argomenti.sh
./funzione_argomenti.sh
```

Output atteso:

```
/etc/hostname: è un file regolare
/etc: è una directory
/file_inesistente: non esiste
Errore: nessun file specificato
```

Nota `return 1`: all'interno di una funzione, `return N` ha lo stesso ruolo di `exit N` nello script — termina la funzione con un codice di uscita. La differenza è che `exit` chiuderebbe l'intero script; `return` chiude solo la funzione e restituisce il controllo al chiamante.

1.3 Variabili locali

Per default, le variabili in Bash sono **globali**: una variabile dichiarata dentro una funzione è visibile anche fuori.

```
cat > variabili_locali.sh << 'EOF'
#!/bin/bash

funzione_test() {
    GLOBALE="sono globale"
    local LOCALE="sono locale"
    echo "Dentro la funzione: GLOBALE=$GLOBALE, LOCALE=$LOCALE"
}

funzione_test
echo "Fuori dalla funzione: GLOBALE=$GLOBALE"
echo "Fuori dalla funzione: LOCALE=$LOCALE"
EOF
chmod +x variabili_locali.sh
./variabili_locali.sh
```

Output atteso:

```
Dentro la funzione: GLOBALE=sono globale, LOCALE=sono locale
Fuori dalla funzione: GLOBALE=sono globale
Fuori dalla funzione: LOCALE=
```

`local` `VARIABILE` limita la visibilità della variabile alla funzione. È buona pratica usare `local` per tutte le variabili di lavoro interne a una funzione — evita di sovrascrivere variabili dello script per errore.

1.4 Restituire un valore stringa

`return` restituisce solo numeri interi (0-255, usati come exit code). Per restituire una stringa da una funzione si usa `echo` + command substitution:

```
cat > funzione_ritorno.sh << 'EOF'
#!/bin/bash

ottieni_estensione() {
    local FILE=$1
    echo "${FILE##*.}"
}

ESTENSIONE=$(ottieni_estensione "documento.pdf")
echo "Estensione: $ESTENSIONE"

ESTENSIONE=$(ottieni_estensione "script.sh")
echo "Estensione: $ESTENSIONE"
EOF
```

```
chmod +x funzione_ritorno.sh
./funzione_ritorno.sh
```

Output atteso:

```
Estensione: pdf
Estensione: sh
```

Il pattern `${FILE##*.}` è una **parameter expansion**: rimuove tutto fino all'ultimo `.` compreso, lasciando solo l'estensione. È introdotto qui come esempio utile — una panoramica completa delle parameter expansion è materiale avanzato.

La funzione esegue `echo` con il risultato, e `$(ottieni_estensione "file.pdf")` cattura quell'output come stringa.

Parte 2 — Il costrutto case

2.1 Struttura base

`case` confronta una variabile con una serie di pattern, uno alla volta. Quando trova il match, esegue i comandi corrispondenti e poi esce dal blocco. È più leggibile di una catena `if/elif/elif/elif` quando i confronti riguardano lo stesso valore.

```
cat > case_base.sh << 'EOF'
#!/bin/bash
GIORNO=$1

case $GIORNO in
    lunedì|martedì|mercoledì|giovedì|venerdì)
        echo "$GIORNO è un giorno lavorativo"
        ;;
    sabato|domenica)
        echo "$GIORNO è un giorno festivo"
        ;;
    *)
        echo "Giorno non riconosciuto: $GIORNO"
        ;;
esac
EOF
chmod +x case_base.sh
./case_base.sh lunedì
./case_base.sh sabato
./case_base.sh pippo
```

Output atteso:

```
lunedì è un giorno lavorativo
sabato è un giorno festivo
Giorno non riconosciuto: pippo
```

Struttura generale:

```

case $VARIABILE in
    pattern1)
        # comandi
        ;;
    pattern2|pattern3)
        # comandi - il / è l'OR tra pattern
        ;;
    *)
        # catch-all - eseguito se nessun pattern ha fatto match
        ;;
esac

```

- ;; chiude ogni branch (obbligatorio)
- esac chiude il blocco case (è “case” al contrario — come fi per if)
- * come pattern corrisponde a qualsiasi valore — usato come default

2.2 Case su estensioni di file

Un uso classico di case in SysAdmin è reagire al tipo di file:

```

cat > case_file.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -z "$FILE" ]; then
    echo "Uso: $0 <file>"
    exit 1
fi

case $FILE in
    *.sh)
        echo "$FILE: script Bash"
        ;;
    *.conf|*.cfg)
        echo "$FILE: file di configurazione"
        ;;
    *.log)
        echo "$FILE: file di log"
        ;;
    *.txt|*.md)
        echo "$FILE: file di testo"
        ;;
    *)
        echo "$FILE: tipo non riconosciuto"
        ;;
esac
EOF
chmod +x case_file.sh
./case_file.sh script.sh

```

```
./case_file.sh /etc/hosts
./case_file.sh syslog.log
./case_file.sh readme.md
```

Il `*` nei pattern qui funziona come wildcard di glob — `*.sh` corrisponde a qualsiasi stringa che termina con `.sh`.

Parte 3 — Test Avanzati e Operatori Logici

3.1 Combinare condizioni con `&&` e `||`

In Modulo 1A hai usato condizioni singole. È possibile combinarne più di una con operatori logici:

```
cat > test_combinati.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -f "$FILE" ] && [ -r "$FILE" ]; then
    echo "$FILE esiste ed è leggibile"
fi

if [ -z "$FILE" ] || [ ! -e "$FILE" ]; then
    echo "Argomento mancante o file inesistente"
fi
EOF
chmod +x test_combinati.sh
./test_combinati.sh /etc/hostname
./test_combinati.sh /file_inesistente
./test_combinati.sh
```

Operatore	Significato
[cond1] && [cond2]	AND — vero se entrambe le condizioni sono vere
[cond1] [cond2]	OR — vero se almeno una condizione è vera
[! cond]	NOT — inverte la condizione

Nota sulla formattazione: `&&` e `||` stanno **fuori** dalle parentesi quadre, non dentro. `[ cond1 && cond2 ]` è sintassi sbagliata in Bash standard. La forma corretta è `[ cond1 ] && [ cond2 ]`.

3.2 Le doppie parentesi `[[ ]]`

Bash offre anche `[[ ]]` (doppia parentesi quadra), una versione estesa e più robusta di `[ ]`:

```
cat > double_bracket.sh << 'EOF'
#!/bin/bash
NOME=$1
```

```

if [[ -n "$NOME" && "$NOME" != root ]]; then
    echo "Utente valido e non root: $NOME"
fi

if [[ "$NOME" == *"admin"* ]]; then
    echo "$NOME contiene la parola admin"
fi
EOF
chmod +x double_bracket.sh
./double_bracket.sh vagrant
./double_bracket.sh sysadmin

```

Differenze principali tra `[ ]` e `[[ ]]`:

Aspetto	<code>[]</code>	<code>[[]]</code>
AND/OR	<code>[c1] && [c2]</code>	<code>[[c1 && c2]]</code> — si può scrivere dentro
Pattern matching	non supportato	<code>[["\$VAR" == *.sh]]</code> funziona
Variabili non quotate	possono dare errori	gestite correttamente
Portabilità	POSIX — funziona ovunque	solo Bash — non funziona in sh puro

Per script Bash (con `#!/bin/bash`) è preferibile usare `[[ ]]`. Per script destinati a essere portabili su shell diverse, usare `[ ]`.

3.3 Leggere l'exit code con `$?`

Ogni comando che esegui in Bash restituisce un exit code. `$?` contiene l'exit code dell'**ultimo comando eseguito**:

```

cat > exit_code.sh << 'EOF'
#!/bin/bash

ls /etc/hostname
echo "Exit code di ls: $?"

ls /file_inesistente
echo "Exit code di ls su file inesistente: $?"

grep "root" /etc/passwd
echo "Exit code di grep (trovato): $?"

grep "utente_che_non_esiste_mai" /etc/passwd
echo "Exit code di grep (non trovato): $?"
EOF

```

```
chmod +x exit_code.sh
./exit_code.sh
```

Output atteso (i codici 0 e 1 si alternano):

```
/etc/hostname
Exit code di ls: 0
ls: cannot access '/file_inesistente': No such file or directory
Exit code di ls su file inesistente: 2
root:x:0:0:root:/root:/bin/bash
Exit code di grep (trovato): 0
Exit code di grep (non trovato): 1
```

Convenzione universale: - 0 = successo - qualsiasi valore diverso da 0 = errore (il valore specifico varia per programma)

\$? deve essere letto **immediatamente** dopo il comando — il comando successivo lo sovrascrive.

Parte 4 — Script completo

Metti insieme funzioni, case, test combinati e exit code in uno script di diagnostica:

```
cat > diagnostica.sh << 'EOF'
#!/bin/bash

# ---- funzioni ----

controlla_servizio() {
    local SERVIZIO=$1
    if systemctl is-active --quiet "$SERVIZIO"; then
        echo " [OK] $SERVIZIO è attivo"
        return 0
    else
        echo " [KO] $SERVIZIO non è attivo"
        return 1
    fi
}

controlla_file() {
    local FILE=$1
    local TIPO

    case $FILE in
        *.conf) TIPO="configurazione" ;;
        *.log)  TIPO="log" ;;
        *.sh)   TIPO="script" ;;
        *)      TIPO="generico" ;;
    esac
```



```

    if [[ -f "$FILE" && -r "$FILE" ]]; then
        echo "    [OK] $FILE ($TIPO) - leggibile"
    elif [ -f "$FILE" ]; then
        echo "    [!] $FILE ($TIPO) - esiste ma non leggibile"
    else
        echo "    [KO] $FILE - non trovato"
    fi
}

# ---- main ----

echo "=== Diagnostica sistema ==="
echo ""

echo ">> Servizi:"
controlla_servizio ssh
controlla_servizio cron

echo ""
echo ">> File critici:"
controlla_file /etc/hostname
controlla_file /etc/ssh/sshd_config
controlla_file /var/log/auth.log
controlla_file /file_inesistente.conf
EOF
chmod +x diagnostica.sh
./diagnostica.sh

```

Leggi lo script funzione per funzione prima di eseguirlo. Identifica: dove viene usato **case**, dove **[[** **]]**, dove **return**, dove viene sfruttato l'exit code implicito di **systemctl**.

Esercizio finale

Scrivi uno script `classifica_dir.sh` che:

1. Riceve una directory come primo argomento (verifica che esista con guard clause)
2. Definisce una funzione `classifica_file` che riceve un percorso e stampa la sua "categoria" tramite **case** sull'estensione
3. Itera su tutti gli elementi della directory con un loop **for**
4. Per ogni elemento, usa **if/elif** per distinguere file da directory, e chiama `classifica_file` solo sui file
5. Alla fine stampa il totale dei file e delle directory trovati

Suggerimento: la struttura è simile ad `analizza_dir.sh` del Modulo 1A, ma con l'aggiunta della funzione `classifica_file` e del **case** al suo interno.

Riepilogo sintattico

Sintassi	Funzione
<code>nome() { ... }</code>	Dichiara una funzione
<code>nome arg1 arg2</code>	Chiama una funzione con argomenti
<code>local VAR=valore</code>	Variabile locale alla funzione
<code>return N</code>	Termina la funzione con exit code N
<code>\$(funzione arg)</code>	Cattura l'output di una funzione come stringa
<code>case \$VAR in p) ;; esac</code>	Confronto su pattern multipli
<code>p1 p2)</code>	OR tra pattern in case
<code>*</code>	Pattern catch-all in case
<code>[c1] && [c2]</code>	AND tra condizioni
<code>[c1] \ \ [c2]</code>	OR tra condizioni
<code>[[c1 && c2]]</code>	AND dentro doppie parentesi
<code>[["\$VAR" == *pattern*]]</code>	Pattern matching in doppie parentesi
<code>\$?</code>	Exit code dell'ultimo comando eseguito

Connessione con Security

Le funzioni sono la struttura portante degli script di enumerazione automatizzata: - Una funzione `scansiona_host()` richiamata in loop su una lista di IP - Un `case` che reagisce diversamente a seconda della porta trovata aperta (22→SSH, 80→HTTP, 443→HTTPS) - `$?` usato per sapere se `nmap` ha trovato qualcosa e decidere se procedere con fasi successive - `[[ ]]` con pattern matching per filtrare output di tool come `nmap` o `netstat`

Collegati

- [\[\[Appunti_modulo1B\]\]](#)
- [\[\[appunti_modulo1B_funzioni_case_test\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[troubleshooting_vm\]\]](#)
· [\[\[metodo_studio_esami_pratici\]\]](#)